

Nếu không thể đo lường, bạn không thể tối ưu.

- Với dữ liệu cân bằng (Balanced data), các lớp xuất hiện đồng đều: Sử dụng **Accuracy** (**Độ chính xác**) và **ROC AUC**.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- (Trong đó: $TP=$ True Positive, $TN=$ True Negative, $FP=$ False Positive, $FN=$ False Negative).

Lựa chọn Độ đo Thành công (Metrics) - Imbalanced Data

Với dữ liệu mất cân bằng nghiêm trọng (vd: 99% giao dịch thường, 1% gian lận), Accuracy vô dụng (đoán bừa "không gian lận" cũng đúng 99%). Ta phải dùng **Precision** và **Recall**.

- **Độ chuẩn xác (Precision):**

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

(Trong số các giao dịch MÁY CẢNH BÁO, bao nhiêu % là gian lận thật?)

- **Độ bao phủ (Recall):**

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

(Trong TỔNG SỐ gian lận có thật, máy tìm ra được bao nhiêu %?)

Chuẩn bị Dữ liệu (Data Preparation)

- **Vector hóa (Vectorization):** Mạng Nơ-ron chỉ nhận Tensor chứa số thực (float32). Mọi chữ viết, âm thanh, hình ảnh đều phải chuyển thành Tensor.
- **Chuẩn hóa (Normalization):** Nếu một đặc trưng chạy từ 0-1, đặc trưng khác chạy từ 100-200 sẽ làm Gradient bùng nổ. Phải đưa về $\mu = 0, \sigma = 1$.

```
1 # Gia su x la ma tran du lieu (samples, features)
2 # Tru di gia tri trung binh (Mean)
3 x -= x.mean(axis=0)
4
5 # Chia cho do lech chuan (Standard deviation)
6 x /= x.std(axis=0)
7
```

Listing 1: Cú pháp Chuẩn hóa cơ bản bằng NumPy

Xử lý Giá trị Khuyết (Handling Missing Values)

Nếu dữ liệu có ô bị trống (Missing):

- **Dữ liệu phân loại (Categorical):** Rất an toàn khi tạo hẳn một mục mới tên là "Khuyết" (Missing). Mô hình sẽ tự học cách xử lý nó.
- **Dữ liệu số (Numerical):** Không nên tự ý điền số 0 (vì có thể làm gián đoạn phân bố). Nên thay bằng **Giá trị Trung bình (Mean)** hoặc **Trung vị (Median)**.
- *Lưu ý:* Nếu dữ liệu Test có thể bị khuyết, mà dữ liệu Train hoàn hảo, hãy cố tình "xóa" bớt một số ô trong tập Train để mô hình tập làm quen với sự thiếu sót.

Chọn Giao thức Đánh giá (Evaluation Protocol)

Làm sao để biết mô hình hoạt động tốt mà không chạm vào tập Test?

- **Hold-out Validation:** Dành cho dữ liệu dồi dào. Cắt hẳn 20% làm Validation.
- **K-Fold Cross-Validation:** Dành cho dữ liệu vừa và nhỏ. Đánh giá chéo qua K vòng lặp để lấy trung bình.
- **Iterated K-Fold:** Lặp lại quy trình K-Fold nhiều lần với các hạt giống ngẫu nhiên (random seeds) khác nhau. Phù hợp cho dữ liệu siêu nhỏ.

Đánh bại Đường cơ sở (Beating a baseline)

- Trước khi xây Deep Learning phức tạp, phải có **Sức mạnh Thống kê (Statistical Power)**. Tức là mô hình phải tốt hơn việc "đoán mò".
- Đường cơ sở (Baseline) có thể là:
 - Dự đoán theo nhãn chiếm đa số.
 - Hoặc một mô hình Học máy cổ điển siêu đơn giản (Random Forest, Logistic Regression).
- **Lưu ý:** Nếu mô hình phức tạp không thắng nổi Baseline đơn giản → Giả thuyết đầu vào chứa đủ thông tin để đoán đầu ra có thể đã **SAI**.

Bảng 6.1: Chọn Activation và Loss function

Bạn không thể tối ưu hóa trực tiếp Accuracy hay ROC AUC (vì nó không khả vi). Phải dùng Proxy Metric là Hàm mất mát (Loss).

Dạng Bài Toán	Lớp Activation cuối	Hàm mất mát (Loss)	Độ đo (Metrics)
Phân loại nhị phân	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
Phân loại đa lớp (1 nhãn)	Softmax	Categorical crossentropy	Categorical accuracy
Phân loại đa lớp (Nhiều nhãn)	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
Hồi quy vô hướng	Không có (None)	Mean squared error (MSE)	Mean absolute error (MAE)

Mở rộng Quy mô (Scaling up: Overfitting Model)

Để tìm ra ranh giới tối ưu, trước tiên bạn phải biết điểm Overfitting nằm ở đâu. Các bước để cố tình tạo ra mô hình Overfit:

- ➊ Thêm các lớp (Layers) cho Mạng Nơ-ron.
- ➋ Tăng số lượng Nơ-ron (Units) ở mỗi lớp để tăng sức chứa (Capacity).
- ➌ Huấn luyện mô hình lâu hơn (tăng số Epochs).

→ Chú ý quan sát Training Loss liên tục giảm, nhưng Validation Loss bắt đầu tăng vọt. Ranh giới lý tưởng nằm ở chính khúc cua này.

Tinh chỉnh và Chuẩn hóa (Regularizing & Tuning)

Đây là bước mất thời gian nhất. Khi đã biết mô hình Overfit ở đâu, ta tiến hành gọt giũa để lấy lại hiệu suất:

- Áp dụng ****Dropout**** để ngắt nơ-ron ngẫu nhiên, làm cho mô hình bớt phụ thuộc vào cục bộ.
- Bổ sung Phạt trọng số ****L1/L2 Regularization****.
- Thử nghiệm các siêu tham số (Learning rate, Batch size, số Units).
- **Cẩn thận:** Tinh chỉnh quá nhiều lần dựa trên tập Validation sẽ dẫn đến việc mô hình học vẹt tập Validation (Information Leakage).

Quản trị Kỳ vọng với Stakeholders

- **Ảo tưởng AI:** Người làm kinh doanh thường hy vọng AI hoàn hảo, hiểu như con người.
- **Thực tế Học máy:** Mô hình chỉ khớp mẫu xác suất, luôn có sai số (False Positives, False Negatives).
- **Kỹ sư phải thiết lập kỳ vọng:** Không nói "Mô hình chính xác 98%". Hãy nói: *"Mỗi ngày máy sẽ bắt đúng 266 giao dịch lừa đảo, chặn nhầm 200 giao dịch thật (cần nhân viên xem lại), và bỏ sót 14 giao dịch lừa đảo"*.
- Lựa chọn Ngưỡng xác suất (Threshold) là sự đánh đổi kinh doanh.

Xuất mô hình (Exporting an inference model)

Bạn không bao giờ chạy Python Object thô trên môi trường Production. Cần xuất ra đồ thị tính toán (Computation Graph).

```
1 # Xuất mô hình dưới dạng artifact TensorFlow SavedModel
2 model.export("path/to/location", format="tf_saved_model")
3
4 # Load lại ở một tiến trình khác, hoặc bằng ngôn ngữ C++
5 reloaded_artifact = tf.saved_model.load("path/to/location")
6 predictions = reloaded_artifact.serve(input_data)
7
```

Listing 2: Xuất mô hình ra TensorFlow SavedModel

```
1 # Export theo chuẩn mô ONNX
2 model.export("path/to/location", format="onnx")
3
4 # Load bằng ONNX Runtime
5 ort_session = onnxruntime.InferenceSession("path/to/location")
6 predictions = ort_session.run(None, input_data)
7
```

Listing 3: Xuất mô hình ra ONNX

Các Môi trường Triển khai (Deployment Environments)

- **REST API (Cloud):** Client gửi yêu cầu HTTP, Server mạnh chạy GPU (TensorFlow Serving).
 - Ưu điểm: Mạnh mẽ. Nhược điểm: Phụ thuộc mạng, độ trễ 500ms, dữ liệu cá nhân bị gửi lên mạng.
- **Trên Thiết bị (On-device / TFLite):** Chạy thẳng vào CPU di động (Android/iOS) hoặc Raspberry Pi.
 - Ưu điểm: Không cần mạng, bảo mật cực cao, không trễ. Nhược điểm: Đòi hỏi mô hình phải nén siêu nhỏ.
- **Trình duyệt (In-browser / TensorFlow.js):** Chạy trên máy tính người dùng bằng JavaScript.
 - Tiết kiệm tiền server, tận dụng GPU cá nhân của khách hàng.

Tối ưu hóa Suy luận (Inference Optimization)

Khi đưa mô hình vào các thiết bị yếu (như điện thoại), cần tối ưu kích thước:

- **Cắt tỉa trọng số (Weight Pruning):** Loại bỏ bớt các kết nối nơ-ron có trọng số quá nhỏ → Tiết kiệm RAM.
- **Lượng tử hóa trọng số (Weight Quantization):** Mạng Nơ-ron vốn lưu số thực chính xác kép (float32). Ta có thể ép nó xuống số nguyên 8-bit (int8).
 - Kích thước giảm ngay **4 lần**.
 - Tốc độ tính toán tăng vọt.
 - Độ chính xác chỉ giảm cực kỳ nhỏ, gần như không đáng kể.
 - Cú pháp: `model.quantize("int8")`

Giám sát trên Môi trường Thực (Monitoring in the wild)

Ngay khi mô hình "Go Live", hiệu suất của nó sẽ bắt đầu suy giảm.

- **A/B Testing Ngẫu nhiên:** Cấp mô hình mới cho 50% user, mô hình cũ cho 50% user. Đo lường Doanh thu / Click-rate để chứng minh sự hiệu quả thực tế.
- **Kiểm toán thủ công (Manual Audit):** Chích xuất 1% dữ liệu thực tế mỗi ngày đưa cho con người gán nhãn thủ công để so sánh với máy.
- **Khảo sát người dùng (User Surveys):** Nút "Báo cáo thư rác bị sai" là nguồn dữ liệu phản hồi (Feedback Loop) tuyệt vời nhất.

Bảo trì và Cập nhật (Maintaining your model)

Không có mô hình nào tồn tại mãi mãi do Concept Drift.

- Vòng đời của hệ thống Gợi ý nhạc có thể chỉ là vài tuần.
- Hệ thống Phát hiện lừa đảo thẻ tín dụng tính bằng Ngày.
- **Chuẩn bị cho thế hệ tiếp theo:**
 - Liên tục thu thập dữ liệu sản xuất.
 - Tìm ra những mẫu *khó nhận* mà mô hình hiện tại đang đoán sai, ưu tiên gán nhãn chúng.
 - Retrain định kỳ → Đây bản cập nhật Version 2.0.